

Homework #2

Athena Mustakis

Problem 1

Use the oil price data either from the dat or csv file to make an informative and aesthetically pleasing plot of oil prices vs. time.

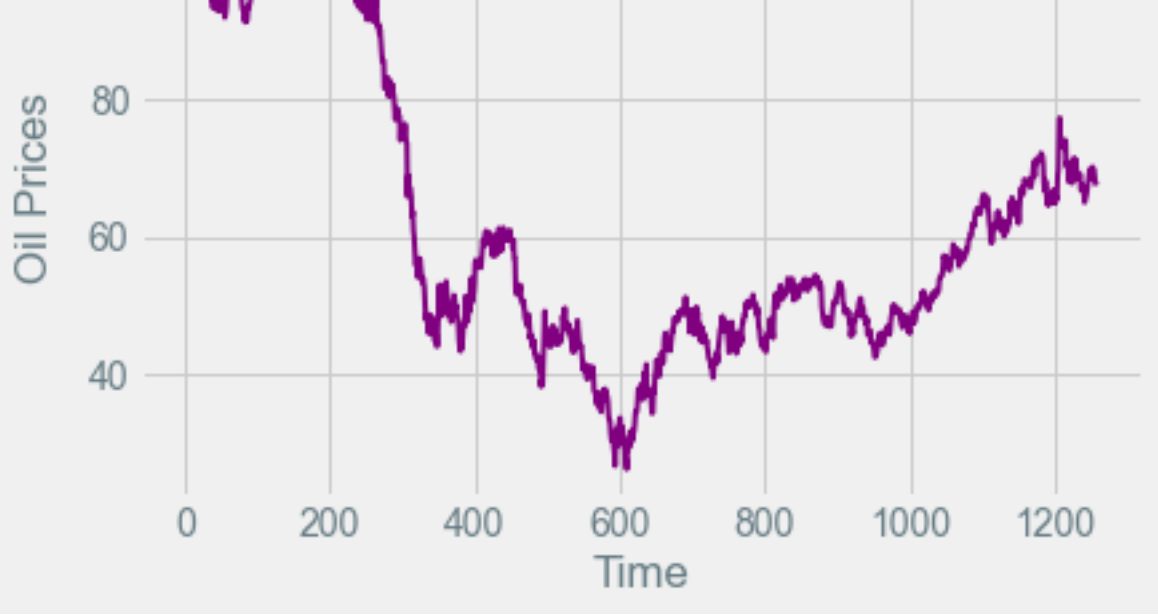
```
In [43]: #get the data in to plot it
import pickle
with open('oil.dat', 'rb') as f:
    dt = pickle.load(f)
print(dt[0:10]) #print out some of the data to look into it

[107.65, 108.72, 108.31, 106.54, 105.36, 108.23, 106.26, 104.7, 103.62, 103.22]

In [44]: #import in the plotting library matplotlib
import matplotlib.pyplot as plt
import numpy as np

In [57]: #creating the time variable
t = list(range(len(dt)))
plt.style.use('fivethirtyeight') #chose a style from style use
plt.plot(dt,color='purple', linewidth=2, linestyle = 'solid')
plt.ylabel('Oil Prices') #add in the x and y axis
plt.xlabel('Time')
plt.title('Oil Prices vs. Time') #add in the title
#list(range(len(oil_data)))

Out[57]: Text(0.5, 1.0, 'Oil Prices vs. Time')
```



Problem 2

This problem follows up on what you did last week with the Human Microbiome Project data. Make a bar plot showing the number of samples for each body site.

Update the previous plot to show the number of samples for male and female subjects. The bars for different genders should be in different color and grouped by body sites.

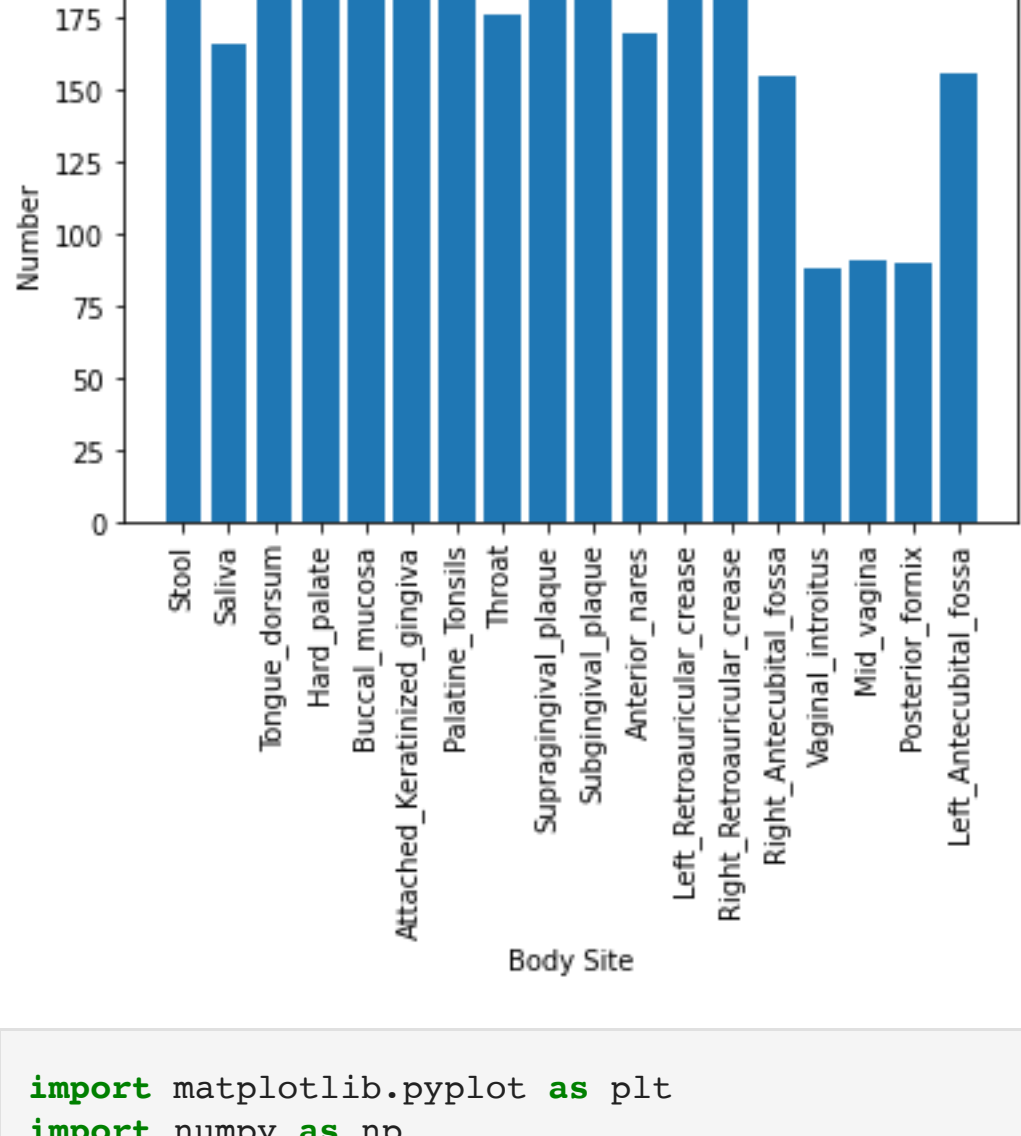
```
In [41]: #need to make a bar plot which is the number of samples for each body site
import matplotlib.pyplot as plt
f = open('v13_map_uniquebyPSN.txt')
data= f.readlines()
d = {} #create an empty dictionary to add to later on
for line in data[1:]: #data line one to the end
    line = line.split() #split the txt files into the columns
    if line[5] not in d: #start making the dictionary that just contains column 5 which is the body site
        d[line[5]] = 1 #if the body site is not already in the dictionary start it at one
    else:
        d[line[5]] += 1 #counts how many times the body site appears in the column
#print(d) #dictionary containing the body sites and the counts
#now turn the dictionary into two lists so that it can be turned into the bar plot
x = list(d.keys())
#print(x)

t = list(d.values())
#print(t)

#now its time to turn it into a bar plot

fig = plt.bar(x,t)
plt.xlabel('Body Site')
plt.ylabel('Number')
plt.title('Number of Samples per Body Site')
plt.xticks(rotation = 90) #rotate the x axis labels so that they are more legiable
plt.show()

#next part is getting the number of samples for each body site
#couldnt you just go through and count how many times each body site shows up? iterate through
```



```
In [8]: import matplotlib.pyplot as plt
import numpy as np
#now update the plot to show number of samples for male and female samples
#bars should be different colors for males and females and should be grouped by body site
#create new lists separated by male and female
import matplotlib.pyplot as plt
f = open('v13_map_uniquebyPSN.txt')
data= f.readlines()
f = {} #create an empty dictionary for just the females
m = {} #create an empty dictionary for just the males

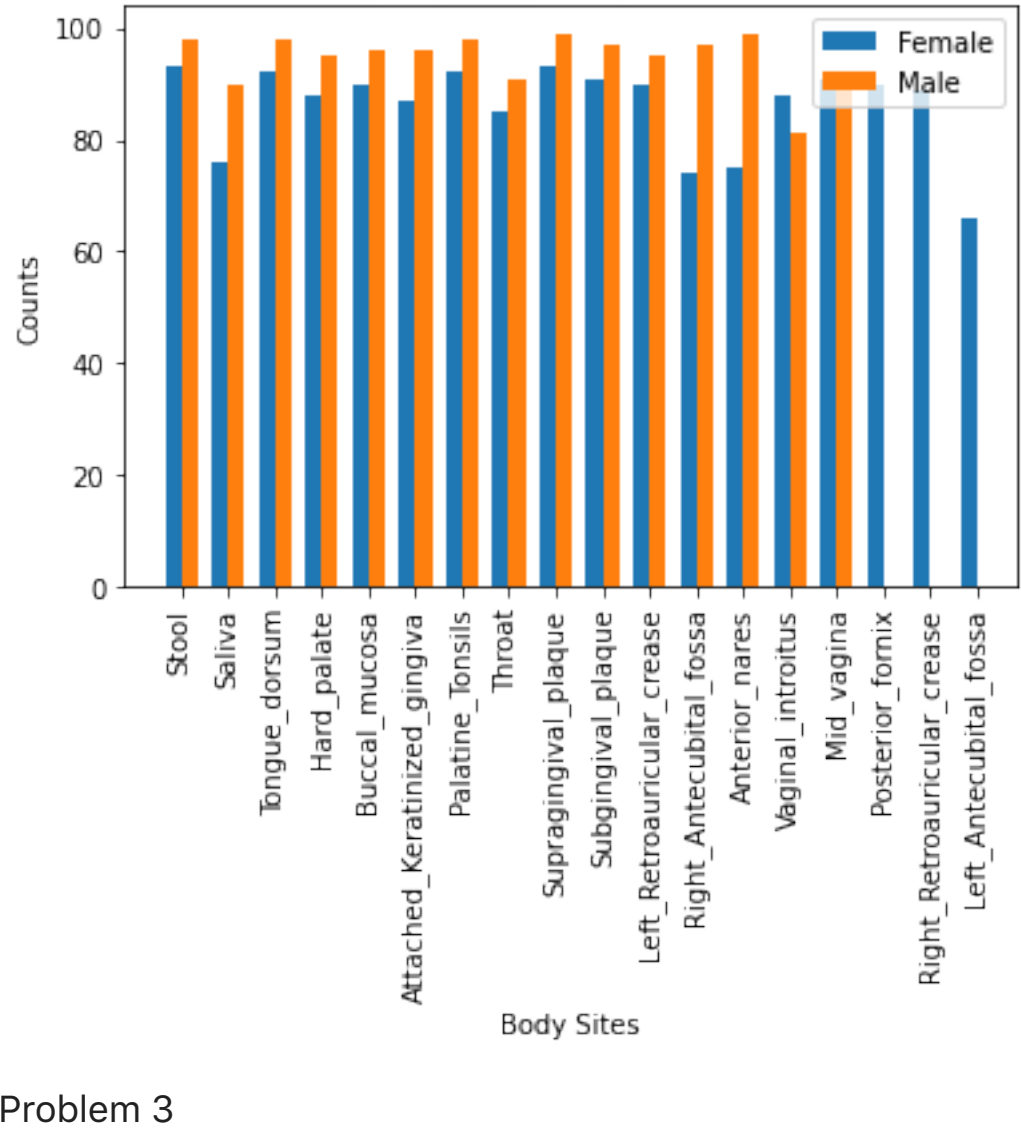
for line in data[1:]:
    line = line.split() #split the line by empty space
    #the third column contains the sex so either male or female
    #make an if statement so we are only looking at the data where it is female and then
    if line[3]== 'female': #if it is female then append that body site to the female list
        if line[5] in f: #if the body site is already in the dictionary just continue the count
            f[line[5]]+=f[line[5]] + 1
        else: #if the body site is not already in the dictionary start the count at one
            f[line[5]] = 1
    else: #do the same process as above with the females but now using males
        if line[5] in m:
            m[line[5]]=m[line[5]] + 1
        else:
            m[line[5]] = 1
print(len(f)) #here the lengths of the dictionary are uneven
print(len(m))

#need to turn the two dictionaries into lists for the plot
femalebody = list(f.keys()) #this is a list containing all the labels
#print(femalebody)
femalecounts = list(f.values())
malebody = list(m.keys())
malecounts = list(m.values())

x = np.arange(len(femalebody)) #set the x axis as the length of the body sites for female and male
x1 = np.arange(len(malebody))
width = 0.35 #set a width of the bars
fig, ax = plt.subplots()
rects1 = ax.bar(x-width/2, femalecounts, width, label = 'Female')
rect2 = ax.bar(x1 + width/2, malecounts, width, label = 'Male')

ax.set_ylabel('Counts')
ax.set_title('Counts grouped by Body Site and Gender')
ax.set_xticks(x) #set the tick marks to be all the body sites
ax.set_xlabel('Body Sites')
ax.set_xticklabels(femalebody, rotation = 90) #rotate the x-label ticks so that it is more readable
ax.legend()

#fig.tight_layout()
plt.show()
```



Problem 3

Download E. coli genome from the course website and use it to simulate a random walk. Assume that 'A' and 'T' correspond to a forward step $\Delta x = +1$ while 'C' and 'G' to a backward step $\Delta x = -1$. Make a plot of "position" x vs. the number of base pairs read/traveled for 3 different initial positions within the genome. Each random walk should travel 100 consecutive base pairs. Let us assume that x = 0 at the start of the random walk. For more information on random walks, see Wikipedia: https://en.wikipedia.org/wiki/Random_walk.

Then, make a histogram of the random walk displacements over one walk for 200 walks; each consecutive random walk starts where the previous one finishes. Finally, combine the two figures made in this problem into a single figure with two subplots arranged horizontally. Add panel labels: "(a)" and "(b)".

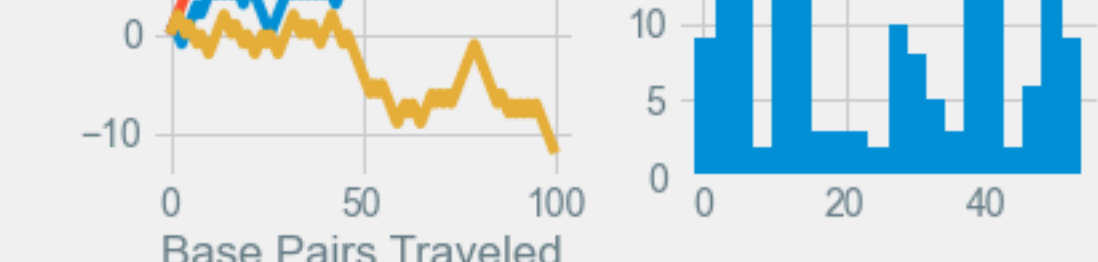
```
In [62]: import matplotlib.pyplot as plt
import numpy as np
import random
#take in the file as an argument
def cleandata(file_name):
    """this function will take in a file read it and then it will replace the new lines and tabs with spaces
    f = open(file_name, 'rt') #remove everything in the file but the sequences
    ln = f.readline() #reads the first line
    dat = f.read() #stores all the data within fasta file not including the header bc header starts with >
    data = dat.replace('\n', ' ').replace('/t', ' ') #get rid of the newlines and tabs in the data
    return data

def randomwalk(data):
    """this function will be used to create the random walks, it will take in the cleaned data and the number of simulations
    simulations = 3 #we want to run the program three times
    steps = 100 #we want it to travel 100 base pairs
    overall_pos = [] #create an empty list to append the values later
    nuc = 0 #initate a counter
    for i in range(simulations):
        pos = [0] #initate the list so that it starts at 0
        for j, nt in enumerate(data[nuc:nuc+steps]): #need to give it the positions to enumerate
            #steps is 100 base pairs, nuc starts at 0 then after one iteration goes to 100 then nuc += 100
            if nt == "A" or nt == "T": #index from the data to get
                pos.append(pos[j] + 1) #add one if its one of these bases
            else:
                pos.append(pos[j]-1) #go back a position if its one of these bases
            nuc += 100
        overall_pos.append(pos) #add the values to the list
    #plot the three lines by indexing the overall_pos list
    return plt.subplot(1,2,1),plt.plot(overall_pos[0]), plt.plot(overall_pos[1]), plt.plot(overall_pos[2])
#row 1 col 2 index 1 for the subplot

def hist(data):
    """this function will be used to create the histogram, it will take in the cleaned data and the number of simulations
    simulations = 1 #we want to run the program 1 time
    steps = 200 #we want it to travel 200 base pairs
    overall_pos = []
    nuc = 0
    for i in range(simulations):
        pos = [0]
        for j, nt in enumerate(data[nuc:nuc+steps]): #need to give it the positions to enumerate
            #steps is 100 base pairs, nuc starts at 0 then after one iteration goes to 100 then nuc += 100
            if nt == "A" or nt == "T": #index from the data to get
                pos.append(pos[j] + 1)
            else:
                pos.append(pos[j]-1)
            nuc += 100
        overall_pos.append(pos)
    return plt.subplot(1,2,2),plt.hist(overall_pos[199], bins = 20), plt.title("Histogram of Random Walk Displacements")
#index 2 for the subplot
#used plt.tight_layout() to format the plots to fit nicely, but both the titles were too long so I used plt.annotate in order to add the panel labels of A and B
c=cleandata('ecoli_genome.fasta')

e=randomwalk(c)
e=hist(c)
#e=randomwalk(c)
#f= hist(c)
```

Ecoli Genome Random Walk Histogram of Random Walk Displacements



```
In [ ]:
```